

STORM MPPI 算法原理

本文档详细说明 STORM (Stochastic Tensor Optimization for Robot Motion) 中 MPPI (Model Predictive Path Integral) 控制器的算法原理。

📖 算法概述

MPPI 是一种基于采样的模型预测控制 (MPC) 算法，源自随机最优控制理论。其核心思想是：

- 从当前动作分布中采样多条轨迹
- 通过前向仿真评估每条轨迹的成本
- 使用指数效用函数加权平均得到最优动作
- 滚动时域执行第一个动作

论文: Williams et al., "Information Theoretic MPC for Model-Based Reinforcement Learning"

📊 算法参数说明 (Algorithm 1 对应)

论文 Algorithm 1 中的符号与代码配置的对应关系：

核心参数

论文符号	含义	默认值	配置项	说明
H	时间步数 (Horizon)	30	<code>mppi.horizon</code>	预测时域长度，30步 × 0.02s = 0.6s
N	采样粒子数 (Num Particles)	500	<code>mppi.num_particles</code>	实际采样 498 + 2特殊粒子
K	优化迭代次数 (Iterations)	1	<code>mppi.n_iters</code>	每个控制周期的 MPPI 优化迭代次数
θ_0	初始动作分布	$\mu=0, \sigma^2=0.005$	<code>mppi.init_cov</code>	初始均值为零，协方差 0.005
M	B样条数据点数 (Knots)	7	<code>knot_scale: 4</code>	$M = H / \text{knot_scale} = 30/4 \approx 7.5$

⚠️ 注意: 论文 Algorithm 1 中的 K 是优化迭代次数，不是 B 样条数据点数！数据点数用 M 表示。

详细参数配置

```
# 文件: examples/HIL/config/ur7e_reacher_hil.yml

mppi:
  # -----
  # 核心参数 (对应论文 Algorithm 1)
  # -----
  horizon: 30          # H: 时间步数 (0.6秒预测窗口)
  num_particles: 500    # N: 采样粒子数
  n_iters: 1           # K: 优化迭代次数 (默认只迭代1次)

  # -----
  #  $\theta_0$ : 初始动作分布参数
  # -----
```

```
init_cov: 0.005      # 初始协方差  $\sigma^2$  (论文中的  $\Sigma$ )
cov_type: 'diag_AxA' # 协方差类型: 对角矩阵 [6×6]
update_cov: False    # 是否动态更新协方差

# -----
# MPPI 算法超参数
# -----
gamma: 0.98          #  $\gamma$ : 折扣因子 (论文中用于成本衰减)
kappa: 0.005         #  $\kappa$ : 协方差正则化参数
beta: 1.0            #  $\beta$ : 动作协方差惩罚系数
alpha: 1             #  $\alpha$ : 控制平滑系数

# -----
# 均值更新参数
# -----
step_size_mean: 0.98  # 均值更新步长:  $\mu_{\text{new}} = 0.98 \mu_{\text{opt}} + 0.02 \mu_{\text{old}}$ 
step_size_cov: 0.7    # 协方差更新步长

# -----
# 采样参数 (B 样条拟合)
# -----
sample_params:
  type: 'multiple'
  sample_ratio: {'halton': 0.0, 'halton-knot': 1.0, ...}
  knot_scale: 4      #  $M = H/\text{knot\_scale} = 30/4 \approx 7$  个数据点
```

参数可视化

Algorithm 1 参数对应关系

论文 Algorithm 1:	代码实现:
Input: $\theta_0 = (\mu_0, \Sigma_0)$	init_cov: 0.005
↓	↓
$\mu_0 = 0$ (零均值)	mean_action 初始化为零
$\Sigma_0 = 0.005 * I$	scale_tril = $\sqrt{0.005} \approx 0.071$
for k = 1 to K:	n_iters: 1 (K=1, 每周期只迭代1次)
↓	↓
优化迭代 K 次	默认只迭代 1 次 (实时性要求)
Sample N trajectories	num_particles: 500
↓	↓
N = 500 条轨迹	498 条 Halton-knot 采样
	+ 1 条零噪声 (均值)
	+ 1 条上次最优
Each trajectory has H steps	horizon: 30
↓	↓
H = 30 步	$30 \times 0.02s = 0.6秒$
B-spline with M knots	knot_scale: 4
↓	↓
M = 7 个数据点	$30 / 4 \approx 7$

💡 为什么 $K=1$ （只迭代一次）？

- 实时控制要求：每个控制周期只有 20ms (50Hz)
- 热启动策略：使用上一周期的解作为初始值，已经是一个好的起点
- 多次迭代会增加计算时间，可能导致控制延迟

INPUT θ_0 详解

论文中的 $\theta_0 = (\mu_0, \Sigma_0)$ 表示**初始动作分布**：

$\theta_0 = (\mu_0, \Sigma_0)$ 初始化

μ_0 （均值动作序列）：

$[H \times A] = [30 \times 6]$ 矩阵，初始值全为 0						
关节 j:	0	1	2	3	4	5
t=0:	0.0	0.0	0.0	0.0	0.0	0.0
t=1:	0.0	0.0	0.0	0.0	0.0	0.0
...	...	...	...	...	...	...
t=29:	0.0	0.0	0.0	0.0	0.0	0.0

Σ_0 （协方差矩阵）：

对于 cov_type='diag_AxA':

$\Sigma_0 = \text{diag}([0.005, 0.005, 0.005, 0.005, 0.005, 0.005])$
= $0.005 \times I_6$

标准差: $\sigma = \sqrt{0.005} \approx 0.071 \text{ rad/s}^2$

含义：每个关节的动作噪声标准差约为 0.071 rad/s²

💡 为什么 $\mu_0 = 0$ ？

- 在第一个控制周期，没有先验信息，假设"不动"是安全的初始猜测
- 之后每个周期会使用上一周期的最优解作为新的均值 (warm-start)

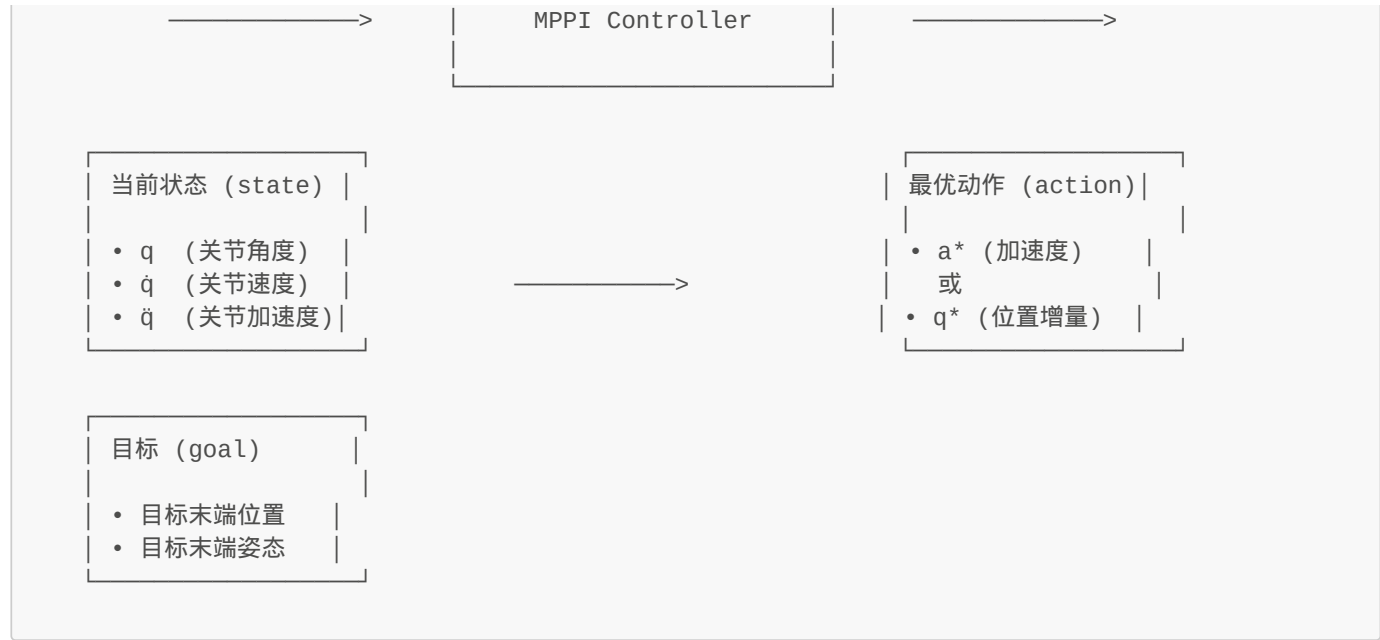
💡 为什么 $\Sigma_0 = 0.005$ ？

- 这个值经过调参，平衡了探索（更大的方差）和稳定性（更小的方差）
- 对于加速度控制，0.071 rad/s² 的标准差提供适度的探索范围

🧠 MPPI 控制器输入与输出

输入





输入详解

1. 当前状态 `state`

```
# 状态向量维度: [n_dofs * 3] = [6 * 3] = 18 维
state = {
    'position':    [q1, q2, q3, q4, q5, q6],      # 关节角度 (rad), 6维
    'velocity':    [v1, v2, v3, v4, v5, v6],      # 关节速度 (rad/s), 6维
    'acceleration': [a1, a2, a3, a4, a5, a6]      # 关节加速度 (rad/s²), 6维
}

# 拼接成完整状态向量
full_state = np.concatenate([position, velocity, acceleration]) # shape: (18,)
```

2. 目标 `goal`

```
# 末端执行器目标
goal_ee_pos = [x, y, z]          # 目标位置 (m), 3维
goal_ee_quat = [qx, qy, qz, qw]  # 目标姿态 (四元数), 4维

# 或者关节空间目标
goal_state = [q1, ..., q6, 0, 0, 0, 0, 0, 0] # 目标关节角度 + 零速度
```

输出详解

最优动作序列 `optimal_action`

```
# MPPI 输出: 最优动作序列
# 维度: [horizon, d_action] = [30, 6]

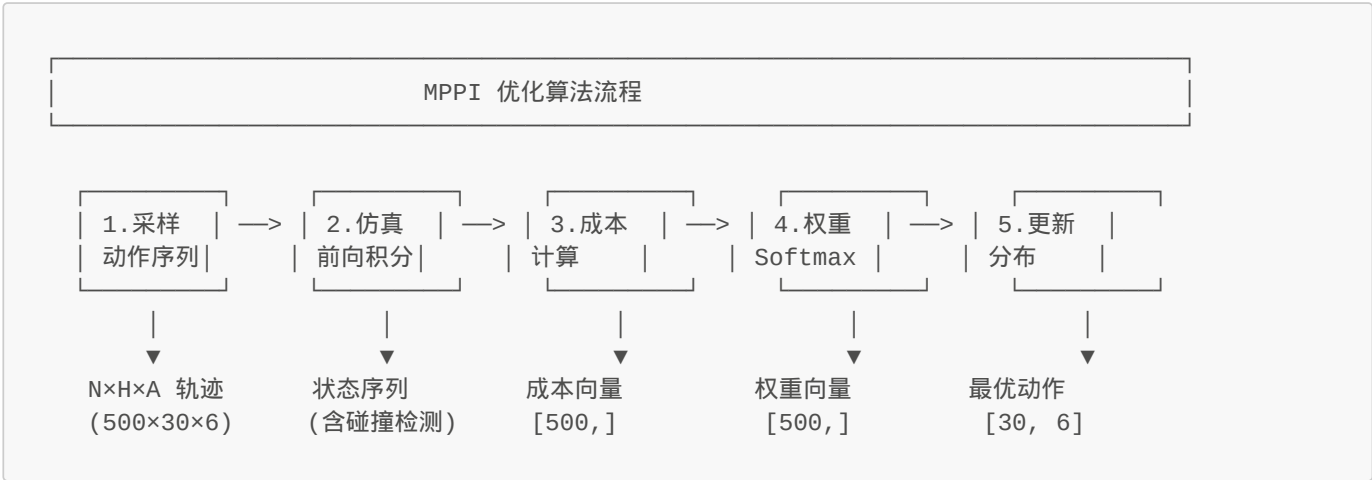
# 控制空间为 'acc' (加速度控制):
optimal_action = [
    [a1_t0, a2_t0, ..., a6_t0], # t=0 时刻的 6 个关节加速度
    [a1_t1, a2_t1, ..., a6_t1], # t=1 时刻
    ...
]
```

```
[a1_t29, a2_t29, ..., a6_t29] # t=29 时刻 (horizon=30)
]

# 只使用第一个动作 (MPC 的滚动时域策略)
action_to_execute = optimal_action[0] # shape: (6,)
```

MPPI 算法流程

总体流程图



各步骤对应源代码文件

步骤	功能	源代码文件 (相对于 storm/)	关键函数
1. 采样	动作序列采样	storm_kit/mpc/control/olgaussian_mpc.py	sample_actions()
	B样条采样库	storm_kit/mpc/control/sample_libs.py	KnotSampleLib.get_samples()
	Halton序列生成	storm_kit/mpc/control/control_utils.py	generate_gaussian_halton_samples()
2. 仿真	前向积分/Rollout	storm_kit/mpc/rollout/arm_base.py	rollout_fn()
	动力学模型	storm_kit/differentiable_robot_model/	URDFKinematicModel.rollout_open_loop()
3. 成本	成本计算	storm_kit/mpc/rollout/arm_base.py	cost_fn()
	各类成本函数	storm_kit/mpc/cost/*.py	PoseCost, CollisionCost, ...
4. 权重	Softmax 权重	storm_kit/mpc/control/mppi.py	_exp_util()

步骤	功能	源代码文件 (相对于 <code>storm/</code>)	关键函数
5. 更新	分布更新	<code>storm_kit/mpc/control/mppi.py</code>	<code>_update_distribution()</code>
	控制器基类	<code>storm_kit/mpc/control/olgaussian_mpc.py</code>	<code>generate_rollouts()</code>

代码调用链



步骤 1: 采样动作序列

从当前动作分布采样 N 条轨迹:

mean_action [H×A]

μ (当前均值)

+

噪声 δ [N×H×A]

$\varepsilon \sim N(0, \Sigma)$

=

采样的动作序列 [N×H×A]

500条轨迹

N=500, H=30, A=6

采样方式 (配置项 `sample_params.type`):

- `multiple`: 混合采样 (推荐, 默认使用)
- `halton`: 纯 Halton 低差异序列
- `random`: 标准高斯随机采样
- `stomp`: STOMP 算法的相关采样

🎮 采样过程详解

采样方式配置

STORM 使用 `MultipleSampleLib` 支持多种采样策略的混合, 配置示例:

```
# 文件: examples/HIL/config/ur7e_reacher_hil.yml
sample_params:
  type: 'multiple'
  fixed_samples: True
  sample_ratio: {'halton': 0.0, 'halton-knot': 1.0, 'random': 0.0, 'random-knot': 0.0}
  seed: 0
  filter_coeffs: None
  knot_scale: 4
```

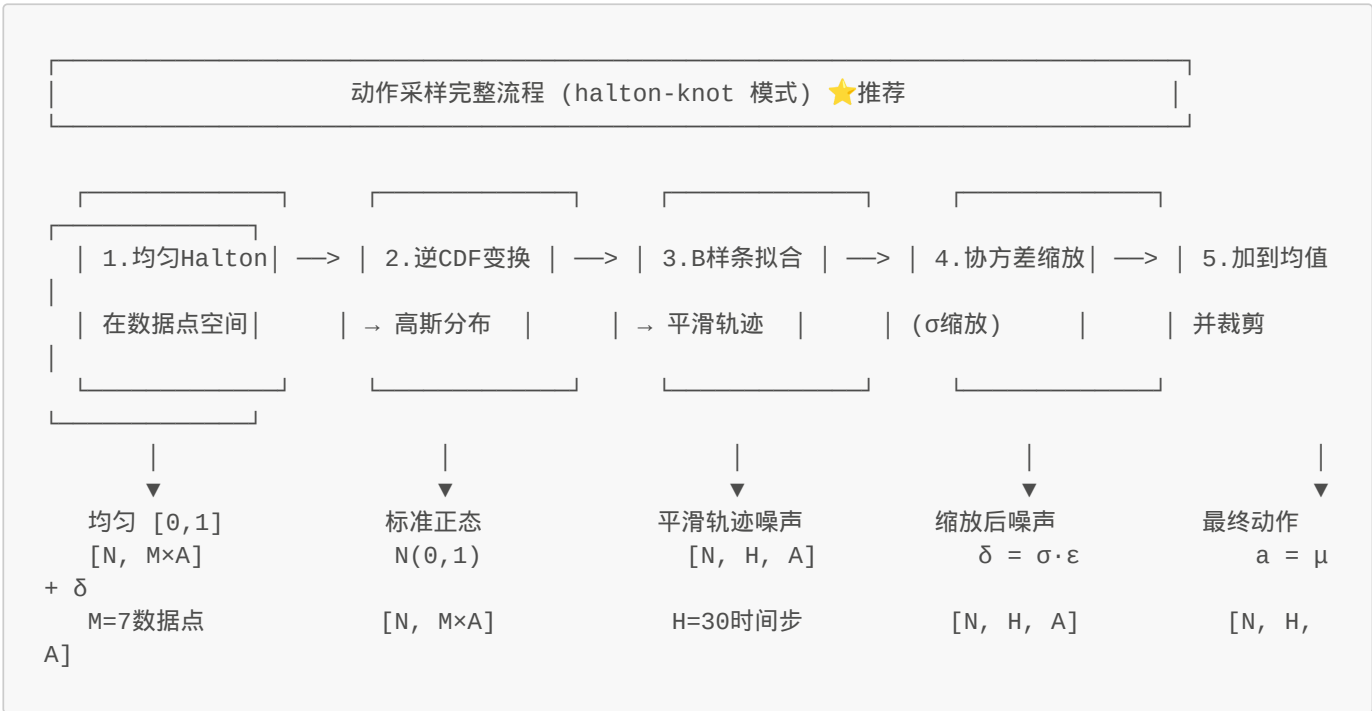
采样比例说明:

采样方法	比例	说明
halton	0.0	纯 Halton 序列采样
halton-knot	1.0	B 样条 + Halton (默认使用)
random	0.0	纯随机高斯采样
random-knot	0.0	B 样条 + 随机采样

⚠️重要: STORM 默认使用 **halton-knot** 采样, 即 **B 样条拟合 + Halton 序列**, 而非纯 Halton 序列!

采样流程总览 (正确流程)

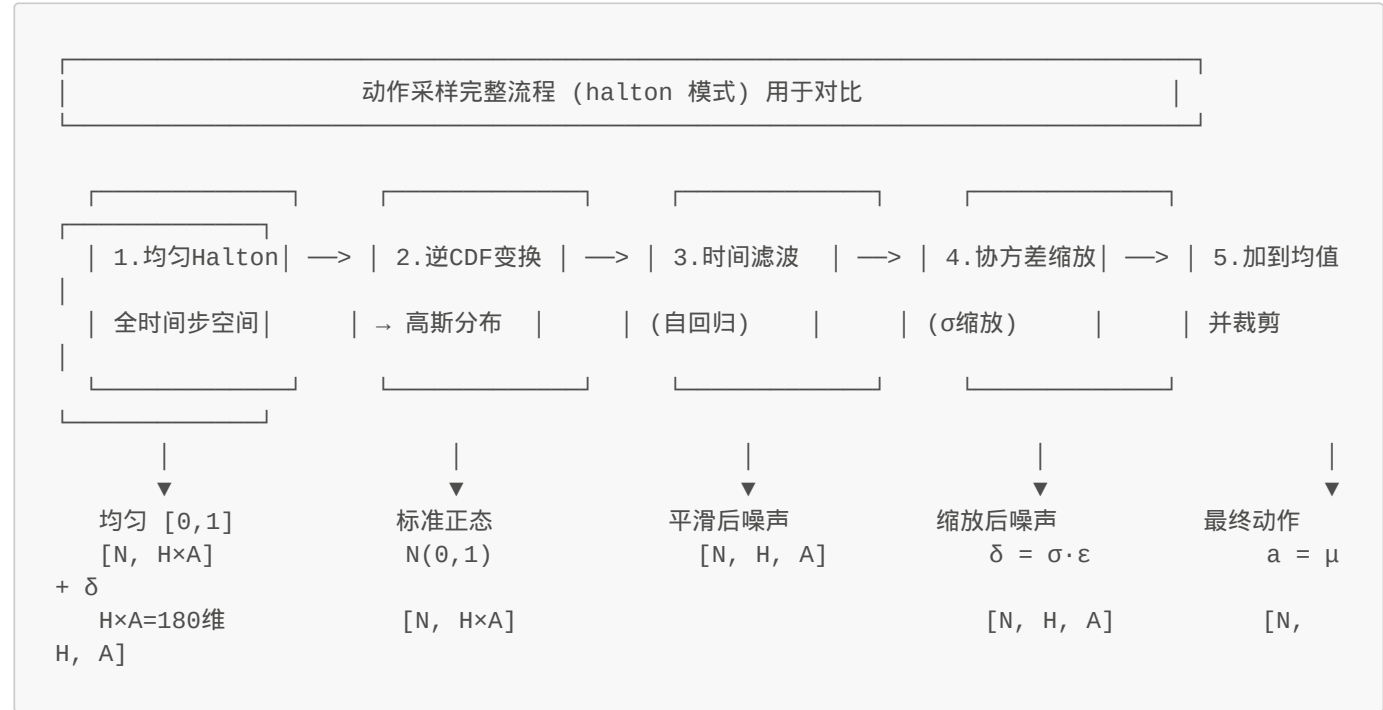
- 📖用户确认: 采样流程是"先生成均匀 Halton, 再逆 CDF 变换, 然后时间滤波, 接着协方差缩放, 加到均值, 裁剪"对吗?
- ✅对于 **halton-knot** 模式 (默认): 流程是 "均匀 Halton → 逆 CDF → B 样条拟合 → 协方差缩放 → 加到均值 → 裁剪"
- 没有时间滤波步骤, 因为 B 样条拟合本身已提供时间平滑性
 - 注意: 7 个采样值是作为**数据点**拟合, 不是直接作为 B 样条控制点!
- ❌对于纯 **halton** 模式: 流程才是 "均匀 Halton → 逆 CDF → 时间滤波 → 协方差缩放 → 加到均值 → 裁剪"



详细步骤说明:

步骤	操作	维度变化	说明
/			

步骤	操作	维度变化	说明
1	Halton 序列采样	$\rightarrow [N, M \times A] = [498, 42]$	在低维数据点空间采样
2	逆 CDF 变换	$[N, 42] \rightarrow [N, 42]$	$\sqrt{2} \cdot \text{erfinv}(2u - 1)$
3	B 样条拟合	$[N, 7, 6] \rightarrow [N, 30, 6]$	7 个数据点拟合 $\rightarrow$ 30 点曲线采样
4	协方差缩放	$[N, 30, 6] \rightarrow [N, 30, 6]$	$\Delta = \sigma \cdot \epsilon$
5	加到均值并裁剪	$[N, 30, 6] \rightarrow [N, 30, 6]$	$a = \text{clamp}(\mu + \Delta)$

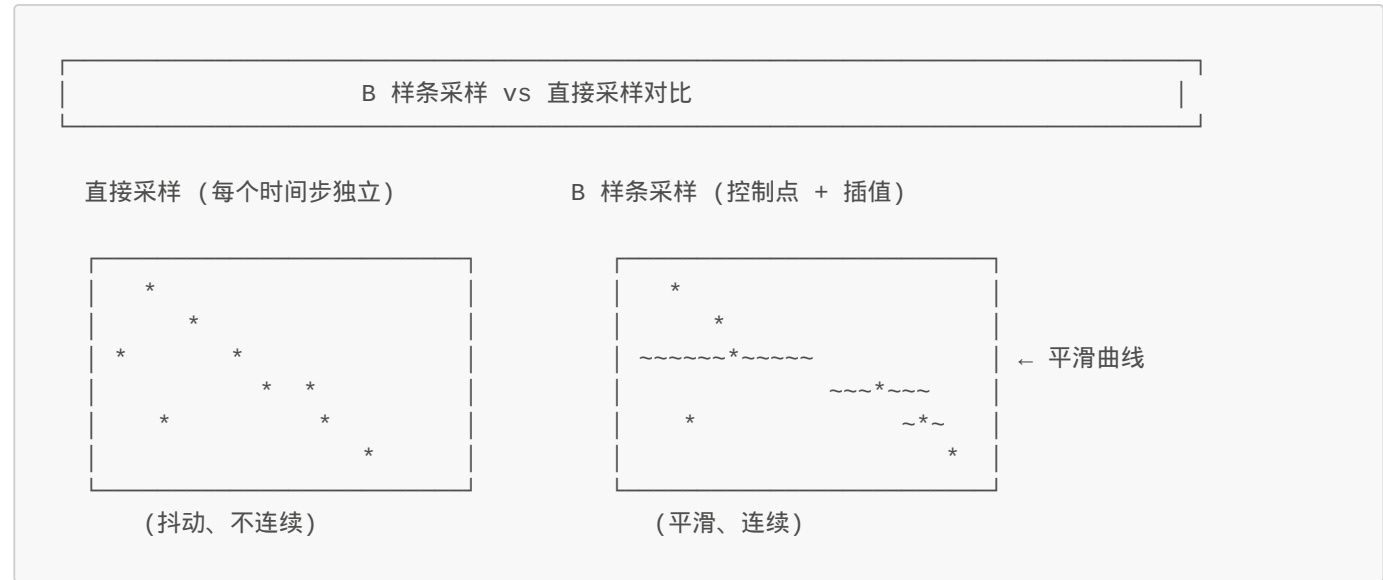


◆ B 样条采样详解 (Knot Sampling)

为什么使用 B 样条？

论文中指出，直接在每个时间步采样的噪声会导致轨迹不平滑、抖动。B 样条采样的核心思想是：

- 在控制点空间采样：**仅采样少量控制点 (knots)
- B 样条插值：**通过 B 样条将控制点插值为完整的平滑轨迹
- 天然平滑性：**B 样条的连续性保证了轨迹的平滑



B 样条采样实现

```
# 文件: storm_kit/mpc/control/sample_libs.py

class KnotSampleLib(object):
    """
    B 样条控制点采样库

    通过在低维控制点空间采样, 然后使用 B 样条插值生成完整轨迹
    """
    def __init__(self, horizon=0, d_action=0, n_knots=0, degree=3,
                  sample_method='halton', **kwargs):
        """
        参数:
            horizon: 时间步数 (30)
            d_action: 动作维度 (6)
            n_knots: 控制点数量 (horizon // knot_scale = 30 // 4 = 7)
            degree: B 样条阶数 (默认 3 次样条)
            sample_method: 'halton' 或 'random'
        """
        self.ndims = n_knots * d_action # 7 * 6 = 42 维
        self.n_knots = n_knots # 7 个控制点
        self.horizon = horizon # 30 个时间步
        self.d_action = d_action # 6 个关节
        self.degree = degree # 3 次 B 样条

    def get_samples(self, sample_shape, **kwargs):
        """
        采样过程:
        1. 采样控制点 (Halton 或 Random)
        2. B 样条插值生成完整轨迹
        """

        # Step 1: 采样控制点
        if self.sample_method == 'halton':
            # 使用 Halton 序列采样控制点
            # 采样维度: N x (n_knots x d_action) = N x 42
            self.knot_points = generate_gaussian_halton_samples(
                sample_shape[0], # 采样数量
                self.ndims, # 42 维 (7个控制点 x 6个关节)
                use_ghalton=True,
                seed_val=self.seed_val,
                device=self.tensor_args['device'],
                float_dtype=self.tensor_args['dtype']
            )
        elif self.sample_method == 'random':
            # 使用随机高斯采样控制点
            self.knot_points = self.mvn.sample(sample_shape=sample_shape)

        # Step 2: 重塑为 [N, d_action, n_knots]
        knot_samples = self.knot_points.view(
            sample_shape[0], # N
            self.d_action, # 6
            self.n_knots # 7
        )

        # Step 3: 初始化输出张量 [N, horizon, d_action]
        self.samples = torch.zeros(
            (sample_shape[0], self.horizon, self.d_action),
            **self.tensor_args
        )

```

```

# Step 4: 对每个粒子的每个动作维度进行 B 样条插值
for i in range(sample_shape[0]):      # 遍历 N 个粒子
    for j in range(self.d_action):      # 遍历 6 个关节
        # 从 7 个控制点插值生成 30 个时间步
        self.samples[i, :, j] = bspline(
            knot_samples[i, j, :],      # 7 个控制点
            n=self.horizon,              # 插值到 30 个点
            degree=self.degree          # 3 次 B 样条
        )

return self.samples # [N, 30, 6]

```

B 样条拟合函数（重要澄清）

⚠ **术语澄清：**代码中的 `bspline()` 函数实际上执行的是 **B 样条拟合/逼近 (Spline Fitting)**，而不是直接把采样点作为 B 样条的控制点 (Control Points)。这是一个重要的区别！

```

# 文件: storm_kit/mpc/control/sample_libs.py

from scipy.interpolate import BSpline
import scipy.interpolate as si

def bspline(c_arr, t_arr=None, n=100, degree=3):
    """
    使用 SciPy 进行 B 样条**拟合**（非直接控制点插值）

    参数:
        c_arr: 数据点数组（7 个采样值，作为拟合的目标点）
        t_arr: 数据点对应的参数值（默认均匀分布 [0, 1, 2, ..., 6]）
        n: 输出采样点数（30）
        degree: 样条阶数（3）

    返回:
        拟合后的平滑曲线上均匀采样的 30 个点
    """
    sample_device = c_arr.device
    sample_dtype = c_arr.dtype
    cv = c_arr.cpu().numpy() # 7 个数据点
    count = len(cv)

    # Step 1: 为 7 个数据点分配参数值 t = [0, 1, 2, 3, 4, 5, 6]
    if t_arr is None:
        t_arr = np.linspace(0, cv.shape[0], cv.shape[0]) # [0, 1.17, 2.33, 3.5, 4.67, 5.83, 7]
    else:
        t_arr = t_arr.cpu().numpy()

    # Step 2: B 样条拟合（关键步骤！）
    # splrep 会自动确定内部节点和控制点，使曲线"逼近"数据点
    # s=0.5 是平滑因子，s>0 表示允许曲线不完全通过数据点
    spl = si.splrep(t_arr, cv, k=degree, s=0.5)
    #
    #           ↑       ↑       ↑       ↑
    #           参数值  数据值  阶数   平滑因子

    # Step 3: 在参数范围 [0, 7] 内均匀采样 30 个点
    xx = np.linspace(0, cv.shape[0], n) # [0, 0.24, 0.48, ..., 7]

    # Step 4: 在拟合的样条曲线上求值

```

```
samples = si.splev(xx, spl, ext=3) # ext=3 表示超出范围返回边界值

samples = torch.as_tensor(samples, device=sample_device, dtype=sample_dtype)

return samples
```

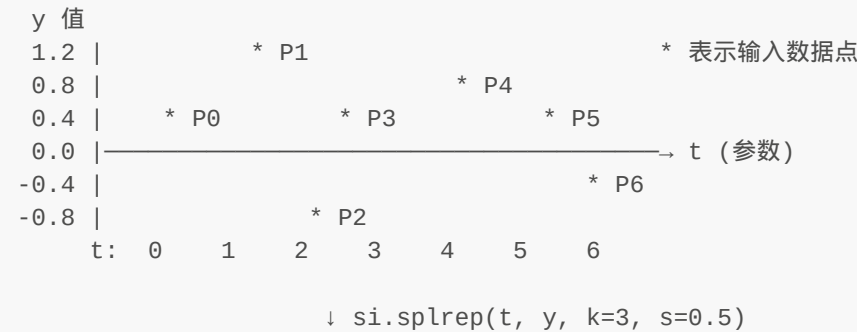
B 样条拟合 vs 控制点插值的区别

这是一个常见的误解，需要澄清：

概念	B 样条控制点插值	B 样条拟合（代码实际使用）
输入含义	控制点 (Control Points)	数据点/采样点 (Data Points)
曲线特性	曲线被控制点"拉"向但不一定通过	曲线尽量"逼近"数据点
节点确定	需要预先指定节点向量	算法自动确定最优节点
平滑参数	无	s 参数控制平滑度
精确通过	只通过首尾控制点	s=0 时精确插值
SciPy 函数	BSpline(t, c, k)	splrep() + splev()

B 样条拟合过程详解（代码实际行为）

输入：7 个数据点（从 Halton + 逆CDF 得到的高斯采样值）



内部处理：

1. 算法自动确定 B 样条的节点向量 (knot vector)

2. 计算 B 样条系数 (coefficients), 使曲线最小化与数据点的加权距离

3. 平滑因子 s=0.5 允许曲线不完全通过数据点（平滑优先）

- s=0 时精确通过所有数据点（纯插值）

- s 越大，曲线越平滑但可能偏离数据点更多

si.splev(xx, spl) 在 30 个点上求值

输出：30 个平滑曲线上的值



为什么使用拟合而非直接控制点？

1. **更灵活**: 不需要预先确定节点向量, 算法自动优化
2. **平滑控制**: 通过 `s` 参数可以控制平滑度
3. **鲁棒性**: 采样点可能有噪声, 拟合可以平滑掉异常值
4. **简单接口**: 直接提供数据点, 无需理解 B 样条内部结构

 **论文对应：**STORM 论文中提到的 "knot points" 实际指的是用于拟合的**稀疏数据点**，而非 B 样条数学定义中的控制点。通过在低维数据点空间采样，再拟合生成高维平滑轨迹。

B 样条拟合数学公式

B 样条曲线的数学定义:

$$S(t) = \sum_{i=0}^n P_i B_{i,k}(t)$$

其中：

- $S(t)$: 时间 t 处的曲线值
- P_i : 第 i 个控制点 (从 Halton 序列采样)
- $B_{i,k}(t)$: k 阶 B 样条基函数
- n : 控制点数量 (7 个)
- k : 样条阶数 (3 次)

采样维度变化:



💡 **优势:** 采样发生在 42 维空间而非 180 维空间，同时 B 样条保证了时间平滑性！

采样方法总结对比

方法	采样空间维度	时间平滑	空间覆盖	计算开销	推荐场景
halton	H×A = 180	❌(需滤波)	✅均匀	中	快速测试
random	H×A = 180	❌(需滤波)	❌随机	低	基线对比
halton-knot	M×A = 42	✅B样条	✅均匀	中	★推荐
random-knot	M×A = 42	✅B样条	❌随机	低	对比实验
stomp	H×A = 180	✅协方差	❌随机	高	特殊场景

关键参数:

- `knot_scale = 4`: 数据点数量 $M = \text{horizon} / \text{knot_scale} = 30 / 4 \approx 7$
- `degree = 3`: B 样条阶数 (3 次样条, C^2 连续)

Halton 序列原理

STORM 在 B 样条控制点采样时使用 Halton 低差异序列：

```
# 文件: storm_kit/mpc/control/control_utils.py

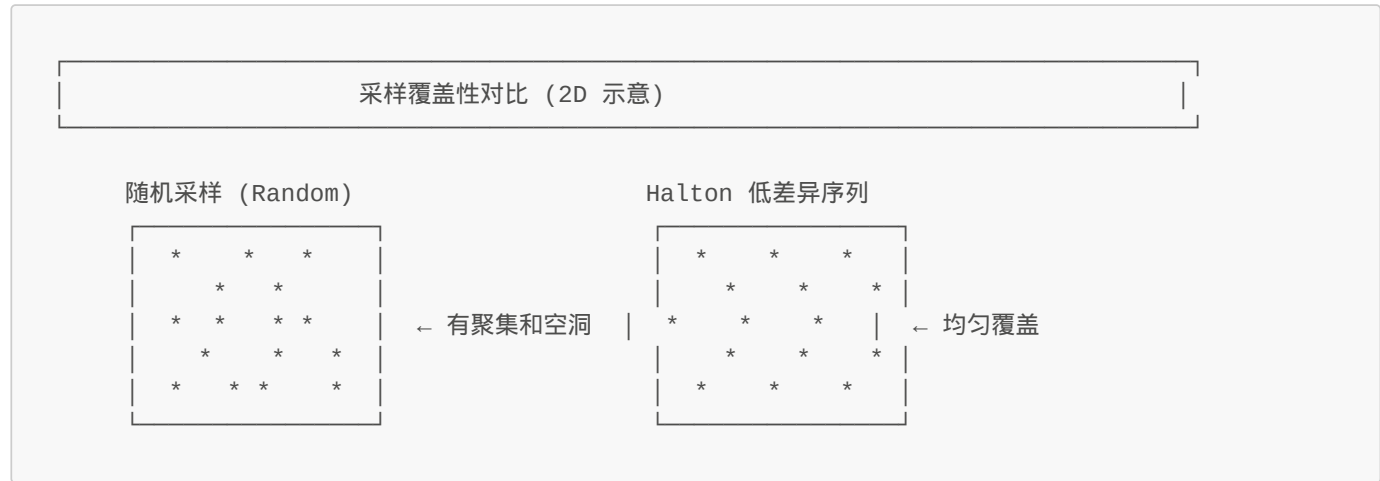
def generate_gaussian_halton_samples(num_samples, ndims, seed_val, device, float_dtype):
    """
    生成高斯分布的 Halton 样本

    Halton 序列是一种准随机序列，比纯随机采样有更好的空间覆盖性
    """
    # 1. 生成均匀分布的 Halton 样本 [0, 1]
    uniform_halton_samples = generate_halton_samples(
        num_samples, ndims, use_ghalton=True, seed_val=seed_val, device=device
    )

    # 2. 通过逆误差函数转换为高斯分布
    # 逆 CDF 变换: U[0,1] -> N(0,1)
    gaussian_halton_samples = sqrt(2) * erfinv(2 * uniform_halton_samples - 1)

    return gaussian_halton_samples # shape: [num_samples, ndims]
```

Halton 序列 vs 随机采样对比:



为什么使用 Halton?

- 更均匀地覆盖控制点空间
- 减少采样数量也能保持良好覆盖
- 确定性序列，便于调试和复现
- 结合 B 样条，在低维空间采样高维平滑轨迹

可选的时间滤波器

当使用纯 `halton` 或 `random` 采样（非 B 样条）时，可选择使用自回归滤波器增加时间平滑性：

```
# 文件: storm_kit/mpc/control/sample_libs.py

def filter_samples(self, eps):
    """
    自回归滤波器，增加时间相关性
```

```
eps[t] =  $\beta_0$  * eps[t] +  $\beta_1$  * eps[t-1] +  $\beta_2$  * eps[t-2]
```

注意：使用 B 样条采样时通常设置 `filter_coeffs: None`，
因为 B 样条本身已保证平滑性

```
"""
if self.filter_coeffs is not None:
    beta_0, beta_1, beta_2 = self.filter_coeffs

    for i in range(2, eps.shape[1]):
        eps[:, i, :] = (beta_0 * eps[:, i, :] +
                        beta_1 * eps[:, i-1, :] +
                        beta_2 * eps[:, i-2, :])

return eps
```

💡 注意: 使用 `halton-knot` 采样时, `filter_coeffs` 通常设为 `None`, 因为 B 样条插值已经保证了轨迹平滑性。

🔧 协方差缩放

B 样条采样后得到的噪声轨迹通过协方差矩阵的 Cholesky 分解进行缩放：

```
# 文件: storm_kit/mpc/control/olgaussian_mpc.py

def sample_actions(self, state=None):
    """
    完整的动作采样函数

    对于 halton-knot 采样模式:
    1. MultipleSampleLib 调用 KnotSampleLib.get_samples()
    2. KnotSampleLib 内部: Halton采样控制点 -> B样条插值 -> 平滑轨迹
    3. 返回的 delta 已经是平滑的噪声轨迹 [N, H, A]
    """

    # Step 1: 从采样库获取噪声轨迹
    # 对于 halton-knot: 返回的是 B 样条插值后的平滑噪声
    # delta: [N-2, H, A] 其中 N-2 = 498
    delta = self.sample_lib.get_samples(
        sample_shape=self.sample_shape, # [498]
        base_seed=self.seed_val + self.num_steps
    )

    # Step 2: 添加零噪声序列 (确保均值动作在采样中)
    # Z_seq: [1, H, A] 全零
    delta = torch.cat((delta, self.Z_seq), dim=0) # [499, H, A]

    # Step 3: 协方差缩放 (可选, 取决于 cov_type)
    # 对于 halton-knot + diag_AxA: 按关节维度缩放
    if self.cov_type == 'diag_AxA':
        # scale_tril: [A] = [6]
        scaled_delta = delta * self.scale_tril # 逐元素相乘
    elif self.cov_type == 'full_HxHA':
        # 完整时空协方差矩阵
        delta = delta.view(delta.shape[0], self.horizon * self.d_action)
        scaled_delta = torch.matmul(delta, self.full_scale_tril)
        scaled_delta = scaled_delta.view(delta.shape[0], self.horizon, self.d_action)
    else:
        scaled_delta = delta * self.scale_tril

    # Step 4: 加到均值动作
    # mean_action: [H, A] = [30, 6]
```

```

# act_seq =  $\mu + \sigma * \delta$ 
act_seq = self.mean_action.unsqueeze(0) + scaled_delta # [N, H, A]

# Step 5: 裁剪到动作范围
act_seq = scale_ctrl(act_seq, self.action_lows, self.action_highs,
squash_fn='clamp')

# Step 6: 添加特殊粒子
append_acts = self.best_traj.unsqueeze(0) # [1, H, A] 上一次最优轨迹

if self.num_null_particles > 0:
    null_act_seqs = torch.zeros(self.num_null_particles, self.horizon,
self.d_action)
    append_acts = torch.cat((append_acts, null_act_seqs), dim=0)

# 最终拼接
act_seq = torch.cat((act_seq, append_acts), dim=0) # [500, H, A]

return act_seq

```

协方差类型详解

STORM 支持多种协方差矩阵类型:

```

# 文件: storm_kit/mpc/control/olgaussian_mpc.py

def reset_covariance(self):
    """初始化协方差矩阵"""

    if self.cov_type == 'sigma_I':
        # 标量协方差:  $\Sigma = \sigma^2 * I$ 
        # 所有维度使用相同方差
        self.cov_action = torch.tensor(self.init_cov) # 标量
        self.scale_tril = torch.sqrt(self.cov_action) #  $L = \sigma$ 

    elif self.cov_type == 'diag_AxA':
        # 对角协方差:  $\Sigma = \text{diag}(\sigma_1^2, \sigma_2^2, \dots, \sigma_A^2)$ 
        # 每个动作维度独立方差 (UR7e 配置使用此类型)
        self.cov_action = torch.tensor([self.init_cov] * self.d_action) # [A]
        self.scale_tril = torch.sqrt(self.cov_action) # [A]

    elif self.cov_type == 'full_AxA':
        # 完整 AxA 协方差矩阵
        # 动作维度之间有相关性
        self.cov_action = torch.diag(torch.tensor([self.init_cov] * self.d_action)) #
[A, A]
        self.scale_tril = torch.linalg.cholesky(self.cov_action) # [A, A]

    elif self.cov_type == 'full_HxHA':
        # 完整 (H*A) x (H*A) 协方差矩阵
        # 时间和动作维度都有相关性
        self.cov_action = torch.diag(torch.tensor(
            [self.init_cov] * (self.horizon * self.d_action)
        )) # [H*A, H*A]
        self.scale_tril = torch.linalg.cholesky(self.cov_action) # [H*A, H*A]

```

协方差类型对比:

类型	矩阵大小	时间相关	动作相关	计算开销	推荐场景
sigma_I	标量	✗	✗	最低	快速原型
diag_AxA	[A] = [6]	✗	✗	低	UR7e 默认
full_AxA	[A×A] = [6×6]	✗	✓	中	关节相关性强
full_HAxHA	[HA × HA] = [180×180]	✓	✓	高	需完整时空相关

💡 **UR7e HIL 配置使用 `diag_AxA`**，因为 B 样条已经提供了时间平滑性，无需协方差矩阵引入时间相关。

完整采样数学公式

对于 B 样条采样 (halton-knot)

完整的采样过程可表示为：

$$\mathbf{a}_i = \boldsymbol{\mu} + \sigma \cdot \text{SplineFit}(\text{InvCDF}(\text{Halton}(\mathbf{z}_i)))$$

展开为：

- 数据点采样:** $\mathbf{D}_i = \text{InvCDF}(\text{Halton}(i))$ ，其中 $\mathbf{D}_i \in \mathbb{R}^{(M \times A)}$ ， $M=7$ 个数据点
- B 样条拟合:** $\boldsymbol{\epsilon}_i = \text{SplineFit}(\mathbf{D}_i) \in \mathbb{R}^{(H \times A)}$ （用 `splrep` 拟合后在 30 点采样）
- 协方差缩放:** $\boldsymbol{\delta}_i = \sigma \cdot \boldsymbol{\epsilon}_i$
- 加到均值:** $\mathbf{a}_i = \boldsymbol{\mu} + \boldsymbol{\delta}_i$

其中：

- $\mathbf{a}_i \in \mathbb{R}^{(H \times A)}$: 第 i 条采样的动作序列 (30×6)
- $\boldsymbol{\mu} \in \mathbb{R}^{(H \times A)}$: 当前均值动作
- σ : 协方差缩放因子 (对于 `diag_AxA` 是 [6] 向量)
- Halton : Halton 准随机序列 [0,1]
- InvCDF : 逆误差函数变换到 $\mathcal{N}(0,1)$
- SplineFit : B 样条拟合 (`si.splrep + si.splev`，平滑因子 $s=0.5$)

⚠ **术语说明:** 这里的 $\mathbf{D}_i$ 是**数据点 (Data Points)**，不是 B 样条的控制点 (Control Points)。`splrep` 会自动确定内部的 B 样条控制点和节点向量。

🎯 具体采样例子：单轨迹单关节

为了更直观地理解采样过程，这里以 **第 42 条轨迹的第 3 个关节（关节索引 $j=2$ ）** 为例，详细展示 `halton-knot` 模式下的完整采样流程。

问题设置

- 轨迹索引: $i = 42$ （共 498 条采样轨迹之一）
- 关节索引: $j = 2$ （UR7e 的第 3 个关节）
- 时间步数: $H = 30$
- 控制点数: $K = 7$ (`knot_scale = 4`, 即 $30/4 \approx 7$)
- 协方差: $\sigma_j = 1.0$ （初始值）
- 当前均值动作: $\mu_{:,j} = [0.1, 0.12, 0.15, \dots, 0.08]$ （30 个时间步）

Step 1: Halton 序列生成（控制点空间）

对于第 42 条轨迹、第 3 个关节，需要生成 7 个控制点的 Halton 值：

控制点索引 k:	0	1	2	3	4	5	6
	↓	↓	↓	↓	↓	↓	↓
Halton 值 u_k :	0.234	0.891	0.456	0.672	0.123	0.789	0.345
	↑						
	均匀分布 $[0, 1]$						
	(基于 Halton 准随机序列, 比纯随机更均匀覆盖)						

 **Halton 序列特点:** 使用不同质数作为基底 (2, 3, 5, 7, ...) , 生成低差异序列, 保证在高维空间的均匀覆盖性。

Step 2: 逆 CDF 变换 (转为高斯分布)

? 为什么要将均匀分布转换为高斯分布?

这是 MPPI 算法的核心需求, 有三个关键原因:

- MPPI 理论基础:** MPPI 基于随机最优控制理论, 假设控制噪声服从高斯分布 $\mathcal{N}(0, \Sigma)$ 。权重计算公式 $w_i \propto \exp(-\frac{1}{\lambda} S_i)$ 的推导依赖于高斯分布的数学性质。
- 探索-利用平衡:** 高斯分布的形状特性使得:
 - 大部分采样集中在均值附近 (利用当前最优解)
 - 少量采样分布在尾部 (探索新区域)
 - 均匀分布则会在整个范围内等概率采样, 浪费计算资源
- 协方差缩放有意义:** 后续的协方差缩放 $\Delta = \sigma \cdot \epsilon$ 只有在 $\epsilon \sim \mathcal{N}(0, 1)$ 时才能正确控制探索范围。均匀分布没有标准差的概念。

Halton + 逆 CDF 的组合优势:

- Halton 序列提供均匀覆盖 (比纯随机采样更高效)
- 逆 CDF 变换保证结果服从高斯分布 (满足 MPPI 理论)
- 两者结合 = 高效 + 理论正确

对每个 Halton 值应用逆误差函数, 将均匀分布转换为标准正态分布:

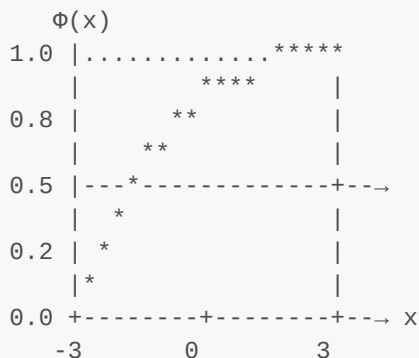
$$\epsilon_k = \sqrt{2} \cdot \text{erfinv}(2 u_k - 1)$$

逆 CDF 变换原理:

逆 CDF 变换 (Inverse Transform Sampling)

标准正态分布的 CDF: $\Phi(x) = P(X \leq x)$

CDF 曲线:



逆变换过程:

给定 $u \in [0, 1]$ (均匀分布)
求 $x = \Phi^{-1}(u)$ 使得 $\Phi(x) = u$

例如: $u = 0.891$
 $x = \Phi^{-1}(0.891) = 1.218$

这个 x 服从 $N(0, 1)$!

数学公式: $x = \sqrt{2} \cdot \text{erfinv}(2u - 1)$ ← 这就是代码中使用的

数据点索引 k:	0	1	2	3	4	5	6
	↓	↓	↓	↓	↓	↓	↓
Halton u_k:	0.234	0.891	0.456	0.672	0.123	0.789	0.345
	↓	↓	↓	↓	↓	↓	↓
2u - 1:	-0.532	0.782	-0.088	0.344	-0.754	0.578	-0.310
	↓	↓	↓	↓	↓	↓	↓
erfinv(.):	-0.507	0.861	-0.078	0.315	-0.774	0.546	-0.287
	↓	↓	↓	↓	↓	↓	↓
ε_k (×√2):	-0.717	1.218	-0.110	0.446	-1.095	0.772	-0.406
	↑						
	标准正态分布 N(0,1)						

现在我们得到 7 个数据点的高斯噪声值： $\boldsymbol{\epsilon}^{\text{data}} = [-0.717, 1.218, -0.110, 0.446, -1.095, 0.772, -0.406]$

Step 3: B 样条拟合（7 个数据点 → 30 个采样点）

⚠ 重要澄清： 这里**不是**直接把 7 个值作为 B 样条的控制点，而是用 B 样条**拟合**这 7 个数据点！

? 什么是"7 个数据点的参数位置"？

在曲线拟合中，每个数据点需要两个信息：

- **参数位置 (t)：** 数据点在曲线上"应该在哪个位置"（类似于时间轴上的位置）
- **数据值 (y)：** 数据点的实际值（我们采样得到的高斯噪声）

参数位置 t:	0	1.17	2.33	3.5	4.67	5.83	7
	↓	↓	↓	↓	↓	↓	↓
数据值 y:	-0.717	1.218	-0.110	0.446	-1.095	0.772	-0.406

为什么需要参数位置？ 因为我们最终要在 30 个时间步上采样，需要知道：

- 这 7 个数据点"代表"轨迹的哪些时间点
- 它们在参数空间 $[0, 7]$ 上均匀分布
- 最终在 $[0, 7]$ 范围内均匀采样 30 个点

拟合过程详解：

```
# 实际代码调用
t_arr = [0, 1.17, 2.33, 3.5, 4.67, 5.83, 7] # 7 个数据点的参数位置（时间轴位置）
cv = [-0.717, 1.218, -0.110, 0.446, -1.095, 0.772, -0.406] # 7 个数据点的值（y 值）

# 注意：np.linspace(0, 7, 7) = [0, 1.167, 2.333, 3.5, 4.667, 5.833, 7]
# 这些是数据点在参数轴上的"位置"，均匀分布在 [0, 7] 范围内

# si.splrep 做的事情：
# 1. 自动确定 B 样条的节点向量（knot vector）
# 2. 计算 B 样条系数，使曲线**逼近**（不一定通过）这些数据点
# 3. s=0.5 是平滑因子，允许曲线不完全通过数据点以获得更平滑的结果
spl = si.splrep(t_arr, cv, k=3, s=0.5)

# 在参数范围 [0, 7] 内均匀采样 30 个点
xx = np.linspace(0, 7, 30) # [0, 0.24, 0.48, ..., 7]
samples = si.splev(xx, spl) # 得到 30 个平滑的输出值
```

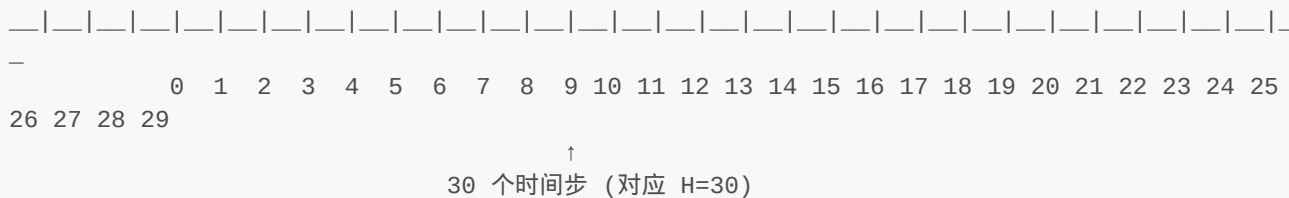
参数轴 t : 0 ————— 7

数据点: * * * * * * *

 (t=0) (t=1.17) (t=2.33) (t=3.5) (t=4.67) (t=5.83) (t=7)

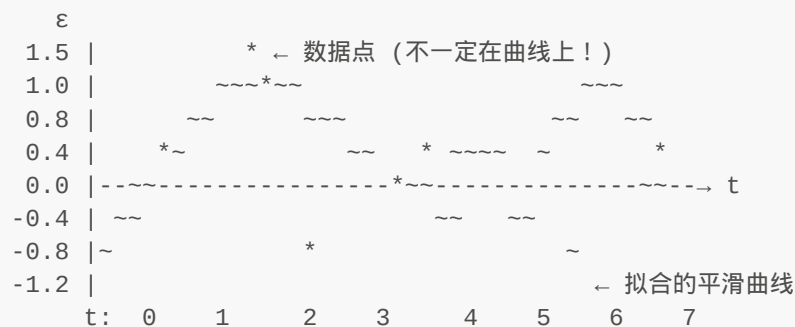
B 样条拟合

输出：



参数映射: $t=0 \rightarrow$ 时间步 0
 $t=7 \rightarrow$ 时间步 29
 $t=3.5 \rightarrow$ 时间步 14.5 (约第 15 步)

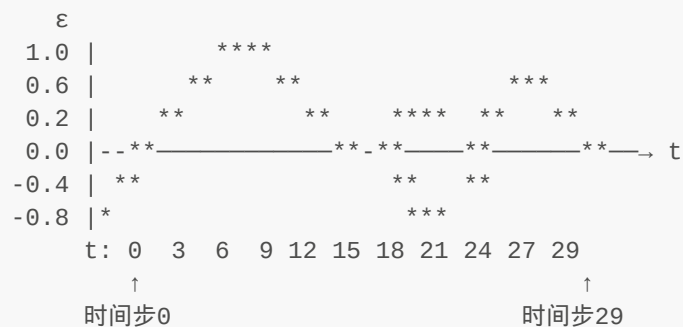
数据点 (M=7) 及拟合曲线:



因为 $s=0.5 > 0$ ，曲线不需要精确通过数据点
这提供了额外的平滑性和抗噪声能力

↓ 在曲线上均匀采样 30 个点

采样后 ($H=30$):



拟合后得到 30 个时间步的平滑噪声值:



ϵ_t : -0.68 -0.49 -0.25 ... 0.38 0.15 ... -0.28 -0.38

↑

注意：与原始数据点 $\epsilon_0=-0.717$ 略有偏差，因为是拟合非插值

💡 关键理解:

- 输入的 7 个值是**数据点**，不是 B 样条的控制点
- `splrep` 会自动计算出真正的 B 样条控制点和节点向量
- 平滑因子 `s=0.5` 使曲线更平滑，但可能不完全通过数据点
- 如果 `s=0`，则变成精确插值，曲线会通过所有数据点

Step 4: 协方差缩放

乘以该关节的标准差（对于 `diag_AxA`，每个关节有独立的 σ_j ）：

$$\delta_t = \sigma_j \cdot \epsilon_t = 1.0 \cdot \epsilon_t$$

时间步 t:	0	1	2	...	14	15	...	28	29
	↓	↓	↓		↓	↓		↓	↓
ϵ_t :	-0.71	-0.52	-0.28	...	0.35	0.12	...	-0.32	-0.41
	$\times \sigma_j = 1.0$								
	↓	↓	↓		↓	↓		↓	↓
δ_t :	-0.71	-0.52	-0.28	...	0.35	0.12	...	-0.32	-0.41

📝 协方差会随着优化过程动态调整，收敛时 σ_j 会减小，探索范围缩小。

Step 5: 加到均值动作

将缩放后的噪声加到当前均值动作上：

$$a_t = \mu_t + \delta_t$$

时间步 t:	0	1	2	...	14	15	...	28	29	
	↓	↓	↓		↓	↓		↓	↓	
μ_t :	0.10	0.12	0.15	...	0.20	0.18	...	0.10	0.08	← 均值动作
	+	+	+		+	+		+	+	
δ_t :	-0.71	-0.52	-0.28	...	0.35	0.12	...	-0.32	-0.41	← 缩放噪声
	=	=	=		=	=		=	=	
a_t :	-0.61	-0.40	-0.13	...	0.55	0.30	...	-0.22	-0.33	← 采样动作

Step 6: 裁剪到动作范围

确保动作在合法范围内（例如 UR7e 关节加速度限制 $[-10, 10]$ rad/s²）：

$$a_t = \text{clamp}(a_t, a_{\text{low}}, a_{\text{high}})$$

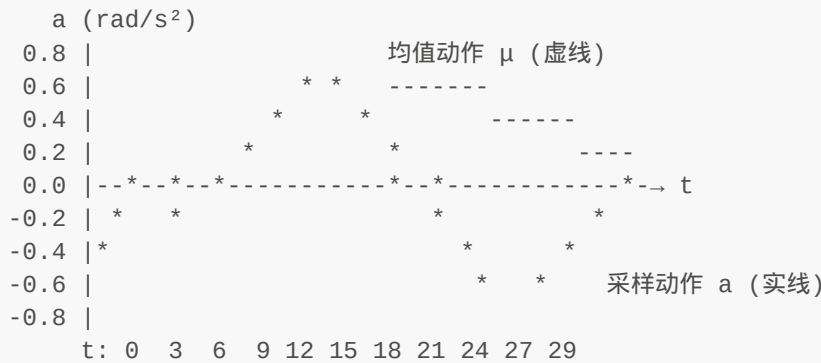
时间步 t:	0	1	2	...	14	15	...	28	29	
	↓	↓	↓		↓	↓		↓	↓	
a_t :	-0.61	-0.40	-0.13	...	0.55	0.30	...	-0.22	-0.33	
	↓ <code>clamp(·, -10, 10)</code>									
a_t :	-0.61	-0.40	-0.13	...	0.55	0.30	...	-0.22	-0.33	← 无变化(均在范围内)

最终结果

对于轨迹 $i=42$ 、关节 $j=2$ ，最终采样得到的 30 步动作序列为：

$$\mathbf{a}_{[42,:]} = [-0.61, -0.40, -0.13, \dots, 0.55, 0.30, \dots, -0.22, -0.33]$$

可视化：采样动作 vs 均值动作



采样过程总结表

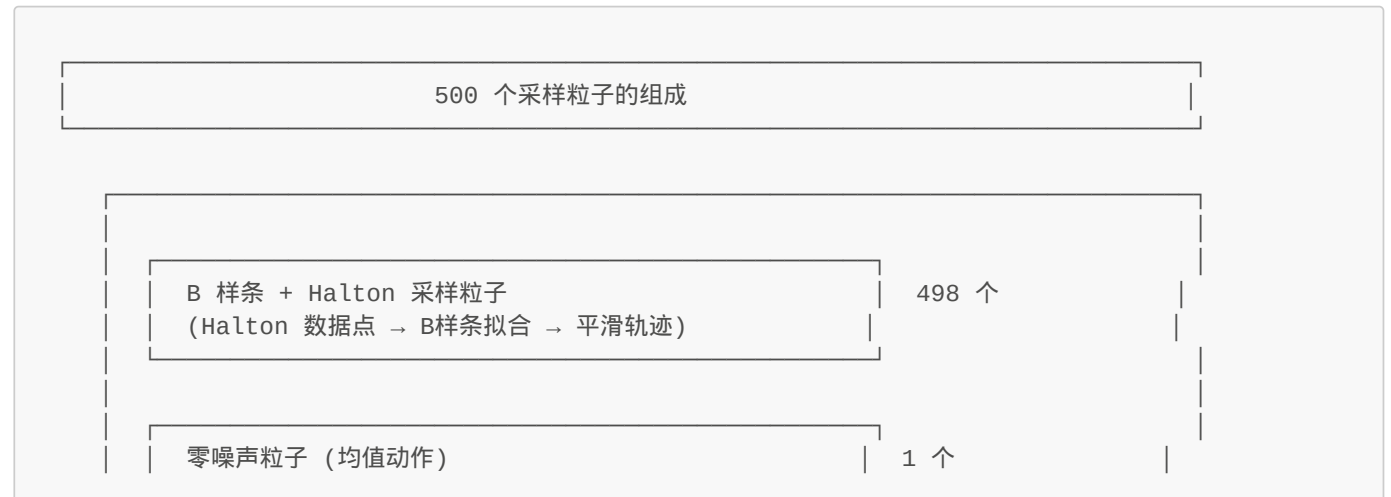
步骤	输入	操作	输出	维度
1	轨迹索引 i , 关节索引 j	Halton 采样	$u_0, \dots, u_6$	7
2	$u_k \in [0,1]$	逆 CDF: $\sqrt{2} \cdot \text{erfinv}(2u-1)$	$\epsilon^{\text{data}}_k \sim N(0,1)$	7
3	7 个数据点	B 样条拟合 (splrep + splev)	30 点平滑曲线 ϵ_t	30
4	ϵ_t	乘 σ_j	$\delta_t = \sigma_j \cdot \epsilon_t$	30
5	δ_t, μ_t	相加	$a_t = \mu_t + \delta_t$	30
6	a_t	裁剪	$a_t = \text{clamp}(a_t)$	30

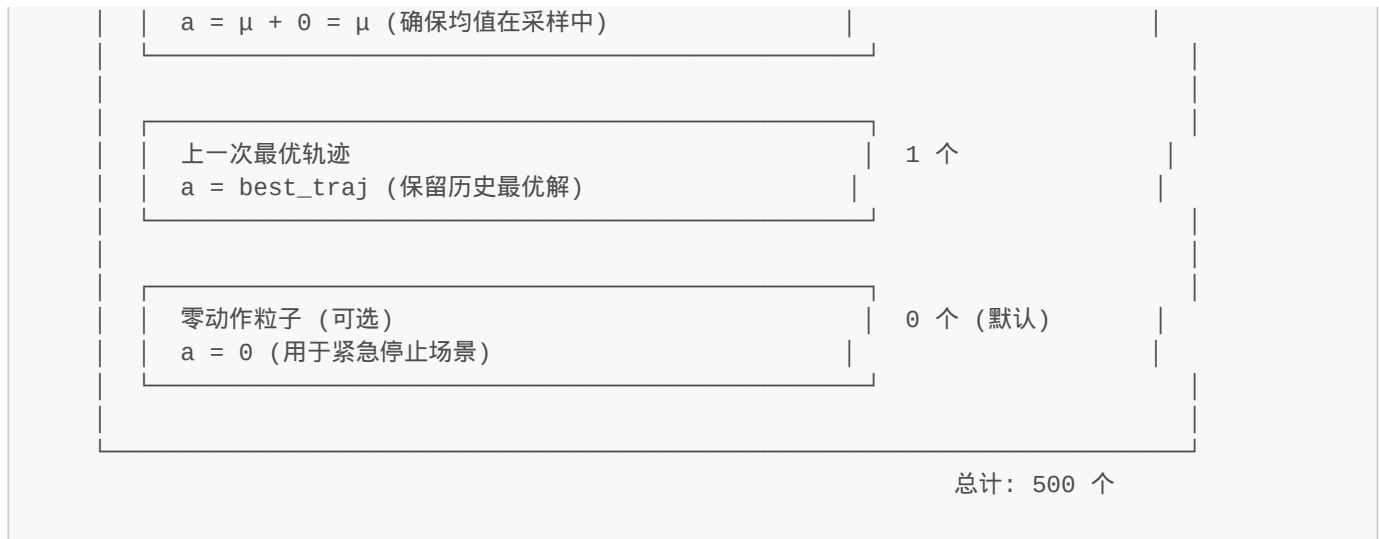
🎯 **关键洞察**：通过在 7 个数据点空间采样（而非 30 个时间步），B 样条拟合**天然保证了轨迹的时间连续性**，这是 STORM 论文的核心创新之一。

⚠️ **术语澄清**：代码变量名 **knot_points** 容易误导，实际上这些是用于拟合的**数据点**，不是 B 样条数学定义中的控制点或节点。

📊 采样组成分析

最终的 500 个采样粒子包含以下组成：





🔗 采样代码调用链 (halton-knot 模式)

```
# 完整的采样调用链 (halton-knot 模式)

# 1. 控制器主循环调用
trajectories = controller.generate_rollouts(state)

# 2. generate_rollouts 内部调用 sample_actions
def generate_rollouts(self, state):
    act_seq = self.sample_actions(state=state) # ← 采样入口
    trajectories = self._rollout_fn(state, act_seq)
    return trajectories

# 3. sample_actions 调用 MultipleSampleLib
def sample_actions(self, state=None):
    # sample_lib 是 MultipleSampleLib 实例
    delta = self.sample_lib.get_samples(...) # ← 获取 B 样条噪声
    ...
    scaled_delta = delta * self.scale_tril # ← 协方差缩放 (diag)
    act_seq = self.mean_action + scaled_delta # ← 加到均值
    act_seq = scale_ctrl(act_seq, ...) # ← 裁剪
    return act_seq

# 4. MultipleSampleLib.get_samples (sample_ratio: halton-knot=1.0)
def get_samples(self, sample_shape, **kwargs):
    # 调用 KnotSampleLib
    samples = self.knot_halton_sample_lib.get_samples(sample_shape)
    return samples

# 5. KnotSampleLib.get_samples (核心采样逻辑)
def get_samples(self, sample_shape):
    # 5a. Halton 采样控制点 (42维 = 7控制点 × 6关节)
    knot_points = generate_gaussian_halton_samples(
        N, self.n_knots * self.d_action, ...
    ) # [N, 42]

    # 5b. 重塑为 [N, d_action, n_knots]
    knot_samples = knot_points.view(N, 6, 7)

    # 5c. 对每个关节进行 B 样条插值
    for i in range(N):
        for j in range(6):
```

```

        samples[i,:,j] = bspline(
            knot_samples[i,j,:], # 7 个控制点
            n=30,                 # 插值到 30 个时间步
            degree=3              # 3 次 B 样条
        )
    return samples # [N, 30, 6]

# 6. bspline (使用 SciPy)
def bspline(c_arr, n, degree):
    spl = si.splrep(t, c_arr, k=degree, s=0.5)
    samples = si.splev(xx, spl)
    return samples

```

步骤 2: 前向仿真 (Rollout)

对每条采样的动作序列，通过动力学模型前向积分：

```

for each particle i in [0, 499]:
    state[0] = current_state
    for t in [0, horizon-1]:
        # 动力学积分 (dt = 0.02s)
        q̈[t] = action[i, t] # 加速度控制
        q̇[t+1] = q̇[t] + q̈[t] * dt
        q[t+1] = q[t] + q̇[t] * dt + 0.5 * q̈[t] * dt2

        # 正运动学：关节空间 → 笛卡尔空间
        ee_pos[t], ee_rot[t] = FK(q[t])
        link_pos[t] = FK_links(q[t])

```

动力学模型:

- 使用 URDF 定义的运动学链
- 可微分正运动学 (PyTorch 实现)
- 批量并行计算 (GPU 加速)

步骤 3: 计算成本

对每条轨迹计算总成本：

Cost = $\sum [$	
$w_{\text{goal}} \times \ ee_pos - goal_pos\ ^2$	目标跟踪
$+ w_{\text{orient}} \times \ ee_rot - goal_rot\ ^2$	姿态跟踪
$+ w_{\text{coll}} \times \text{collision_cost}(\text{link_pos})$	碰撞避免
$+ w_{\text{self}} \times \text{self_collision_cost}(q)$	自碰撞
$+ w_{\text{smooth}} \times \ \Delta \ddot{q}\ ^2$	平滑性
$+ w_{\text{bound}} \times \text{bound_violation}(q, \dot{q}, \ddot{q})$	关节限制
$+ w_{\text{vel}} \times \ \dot{q}\ ^2 \text{ (near goal)}$	到达时减速
$]$	

输出: `costs: [500,]` 每条轨迹一个标量成本

步骤 4: 计算权重 (Softmax)

使用指数效用函数将成本转换为权重：

```
w_i = softmax( -cost_i / β )  
  
      = exp(-cost_i / β) / Σ exp(-cost_j / β)  
  
β (temperature): 控制权重分布的锐度  
- β 小: 只有最优轨迹有高权重 (贪婪)  
- β 大: 权重更均匀 (探索性)
```

输出: `weights`: [500,] 每条轨迹一个权重, $\sum w_i = 1$

步骤 5: 加权平均得到最优动作

```
optimal_action = Σ w_i × action_i  
  
(加权平均所有采样的动作序列)
```

输出: `optimal_action`: [30, 6] 最优动作序列

步骤 6: 积分得到位置指令

```
# 使用第一个动作  
a_cmd = optimal_action[0]      # [6,] 加速度  
  
# 积分得到期望位置  
q_cmd = q_current + q_current * dt + 0.5 * a_cmd * dt^2  
  
# 或直接用 MPC 预测的下一步位置  
q_cmd = predicted_state[1, :6]
```

输出: `q_cmd`: [6,] 发送给机械臂的关节位置指令

 成本函数详解

成本函数组成

成本项	公式	作用	权重 (典型值)
<code>goal_pose</code>	$\ p_{\{ee\}} - p_{\{goal\}}\ ^2 + w_r \ R_{\{ee\}} - R_{\{goal\}}\ ^2$	目标跟踪	15.0, 100.0
<code>primitive_collision</code>	$\sum_i \max(0, r_i - d_i)^2$	障碍物碰撞	5000.0

成本项	公式	作用	权重 (典型值)
robot_self_collision	$\sum_{ij} \max(0, r_{ij} - d_{ij})^2$	自碰撞	5000.0
smooth	$\ \ddot{q}_{t+1} - \ddot{q}_t\ ^2$	轨迹平滑	1.0
state_bound	$\max(0, q - q_{\max})^2 + \max(0, q_{\min} - q)^2$	关节限制	1000.0
zero_vel	$\ \dot{q}\ ^2$ (near goal)	到达减速	0.1
null_space	$\ q - q_{\text{retract}}\ ^2$	冗余优化	0.1

目标跟踪成本

$$C_{\text{goal}} = w_p \|p_{\text{ee}} - p_{\text{goal}}\|^2 + w_r \|R_{\text{ee}} - R_{\text{goal}}\|^2$$

其中:

- p_{ee} : 末端执行器位置
- R_{ee} : 末端执行器旋转矩阵
- $\ominus$: 旋转误差运算

碰撞成本

$$C_{\text{coll}} = \sum_i \max(0, r_i + r_{\text{obs}} - d_i)^2$$

其中:

- r_i : 机器人连杆碰撞球半径
- r_{obs} : 障碍物半径
- d_i : 碰撞球到障碍物中心距离

💻 代码对应关系

MPPI 核心类

```
# 文件: storm_kit/mpc/control/mppi.py

class MPPI(OLGaussianMPC):

    def _update_distribution(self, trajectories):
        """步骤 4-5: 计算权重并更新分布"""
        costs = trajectories["costs"]
        actions = trajectories["actions"]

        # 步骤 4: Softmax 计算权重
        w = self._exp_util(costs, actions)

        # 步骤 5: 加权平均
        weighted_seq = w.unsqueeze(-1).unsqueeze(-1) * actions
        new_mean = torch.sum(weighted_seq, dim=0)

        # 更新均值 (带步长)
        self.mean_action = (1-α) * self.mean_action + α * new_mean

    def _exp_util(self, costs, actions):
        """指数效用函数 (Softmax)"""
        traj_costs = cost_to_go(costs, self.gamma_seq)[: , 0]
        w = torch.softmax((-1.0/self.beta) * traj_costs, dim=0)
        return w
```

Rollout 函数

```
# 文件: storm_kit/mpc/rollout/arm_base.py

class ArmBase(RolloutBase):

    def rollout_fn(self, start_state, act_seq):
        """步骤 2-3: 前向仿真 + 成本计算"""

        # 步骤 2: 动力学前向积分
        state_dict = self.dynamics_model.rollout_open_loop(start_state, act_seq)

        # 步骤 3: 计算成本
        cost_seq = self.cost_fn(state_dict, act_seq)

        return {
            'actions': act_seq,
            'costs': cost_seq,
            'ee_pos_seq': state_dict['ee_pos_seq'],
            ...
        }
```

采样函数

```
# 文件: storm_kit/mpc/control/olgaussian_mpc.py

class OLGaussianMPC(Controller):

    def sample_actions(self, state=None):
        """步骤 1: 采样动作序列"""
        # 生成噪声
        delta = self.sample_lib.get_samples(sample_shape=self.sample_shape)

        # 缩放噪声
        scaled_delta = torch.matmul(delta, self.full_scale_tril)

        # 添加到均值
        act_seq = self.mean_action.unsqueeze(0) + scaled_delta

        # 裁剪到动作范围
        act_seq = scale_ctrl(act_seq, self.action_lows, self.action_highs)

        return act_seq
```

完整控制循环

```
# 伪代码: 完整的 MPC 控制循环

while running:
    # 1. 获取当前状态 (输入)
    state = robot.get_state()
    full_state = np.concatenate([
        state['position'],      # q
```

```
        state['velocity'],      #  $\dot{q}$ 
        state['acceleration']  #  $\ddot{q}$ 
    ])

    # 2. MPPI 优化 (核心计算)
    for _ in range(n_iters): # 通常 1-3 次迭代
        # 步骤 1: 采样动作序列
        actions = mpc.sample_actions(full_state) # [500, 30, 6]

        # 步骤 2-3: 前向仿真 + 成本计算
        trajectories = mpc.rollout_fn(full_state, actions)

        # 步骤 4-5: 更新动作分布
        mpc._update_distribution(trajectories)

    # 3. 获取最优动作 (输出)
    optimal_action = mpc.mean_action[0] # 第一步动作

    # 4. 积分得到位置指令
    dt = 0.02
    q_cmd = state['position'] + state['velocity'] * dt + 0.5 * optimal_action * dt**2

    # 5. 安全限制
    q_cmd = apply_velocity_limit(q_cmd, max_vel=0.5)
    q_cmd = apply_smoothing(q_cmd)

    # 6. 发送到机械臂
    robot.send_position_command(q_cmd)

    # 7. Hotstart: 移动时域窗口
    mpc._shift(shift_steps=1)

    # 8. 等待下一个控制周期
    rate.sleep() # 50Hz
```

⚙️ 关键参数

MPPI 参数

参数	含义	典型值
horizon	预测步数	30
num_particles	采样粒子数	500
beta	温度参数	0.01
gamma	折扣因子	0.99
n_iters	迭代次数	1
step_size_mean	均值更新步长	1.0

动力学参数

参数	含义	典型值
dt	仿真时间步	0.02s
control_dt	控制周期	0.02s

参数	含义	典型值
max_acc	最大加速度	5.0 rad/s²

性能优化

GPU 加速

STORM 使用 PyTorch 在 GPU 上批量并行计算：

- 500 条轨迹同时前向积分
- 批量正运动学计算
- 批量碰撞检测

Hotstart

每个控制周期不从零开始优化，而是使用上一步的解作为初始猜测：

```
# 移动时域窗口
self.mean_action = self.mean_action.roll(-1, 0)
self.mean_action[-1] = 0 # 或重复最后一个动作
```

参考资料

- [STORM MPC 论文](#)
- [MPPI 原始论文](#)
- [STORM GitHub](#)